# A Domain-Specific Language for Microservices

Jacob Donham
jdonham@twitter.com
Twitter Inc.
San Francisco, CA, United States

## Abstract

A common architectural pattern for complex online systems is a collection of "microservices" communicating via RPC interfaces. A service architecture provides autonomy for teams to develop, deploy, and operate system modules without centralized coordination. But this autonomy comes at a cost in leverage—the fixed costs of running a service are high, and the desire to avoid them can lead to poor design choices.

In this paper we describe Strato, a system built at Twitter that runs many microservices hosted in a shared service platform—trading a little autonomy for a lot of leverage. Hosted microservices are written in a Scala-like DSL that supports transparent concurrency and native access to Thrift data (the common currency of services at Twitter). The DSL compiles to a concurrency library written in Scala with an arrows-based interface. We detail the design and implementation of the DSL and concurrency library, and evaluate the effectiveness of Strato in improving leverage at Twitter.

***CCS Concepts*** • **Applied computing → Service-oriented architectures**; • **Software and its engineering → Functional languages**; **Concurrent programming structures**; **Domain specific languages**;

***Keywords*** Scala, microservices

**ACM Reference Format:**
Jacob Donham. 2018. A Domain-Specific Language for Microservices. In *Proceedings of the 9th ACM SIGPLAN International Scala Symposium (Scala '18), September 28, 2018, St. Louis, MO, USA.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3241653.3241654

## 1 Introduction

Like many other large, complex online systems, the Twitter consumer product is implemented as a constellation of independent *services* communicating via Thrift [1] RPC interfaces. There are many hundreds of services running in production at Twitter; handling a typical top-level request (e.g. from a user's browser) involves calling tens of different services and hundreds of service instances.

This arrangement is driven by the following question:

> *With hundreds of engineers working on one system, how does anyone get anything done?*

From this point of view, a services architecture is appealing because it gives engineering teams autonomy:

- each service has its own code base, so teams may develop them without central coordination;
- each service is independently deployed, so teams may operate them without central coordination; and
- each service has an owning team, so domain knowledge and responsibility are focused.

However, services have high fixed costs—common costs to develop, maintain, and operate each service, independent of the application logic it contains. For example:

- setting up the RPC service framework;
- continuous integration and deployment;
- monitoring, alerting, logging, and tracing;
- authorization and access control;
- scale and capacity management; and
- operations and on-call.

At Twitter there are many sophisticated libraries, frameworks, and subsystems to address these needs; still it is up to each service owner to assemble the pieces into a functioning service, and to maintain and operate it over time. These costs reduce teams' leverage—that is, productivity gained by reusing systems or capabilities.

Often the application logic of a service is modest and does not justify this overhead; it's therefore pragmatic to build new functionality into an existing service rather than bring up a new one. As more and more functionality is built into a service, it becomes a "monolith", obviating the advantages of a service architecture: a monolithic service must be centrally developed, deployed, and operated; and ownership is diffuse. Teams working in a monolithic service enjoy greater leverage by reusing its facilities, but lose a lot of autonomy.

In order to improve engineering teams' leverage without taking too much of their autonomy, Twitter is building a new system, called *Strato*, which runs many microservices in a shared service platform. The Strato team is responsible for most of the fixed costs of developing and owning a service, while application teams are responsible for developing their

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
*Scala '18, September 28, 2018, St. Louis, MO, USA*
© 2018 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-5836-1/18/09.
https://doi.org/10.1145/3241653.3241654

application logic. Teams write application logic in an (external, rather than embedded) domain-specific language called *StratoQL*.

StratoQL is a Scala-like language with some unusual features to support the domain of microservices at Twitter: transparent concurrency to support calling other services without syntactic overhead, and a structural type system to support native access to Thrift data.

The contributions of this paper are the following:

- a description of StratoQL's type system and typechecking algorithm (Section 3)
- a description of a concurrency library based on arrows [8] called *Stitch*, and how StratoQL is compiled to this library (Section 4)
- a report on how well Strato has achieved the goal of improving engineering teams' leverage without taking too much of their autonomy (Section 5)

## 2 The Strato Service Platform

For context we briefly sketch the Strato service platform:

Like most services at Twitter, Strato is written in Scala and uses the Finagle library [16] to implement its Thrift RPC client and server stacks; it runs in Twitter's datacenters on top of Aurora/Mesos [3] [4]; it uses various homegrown and open-source libraries and systems for continuous integration, deployment, monitoring, alerting, logging, tracing, and so on; and we typically deploy the system twice a week.

Unlike most services at Twitter, the Strato codebase contains no application logic: microservices running in Strato are defined by configuration files, which are deployed separately from the Strato service. Configuration deployment is a lightweight process compared to service deployment: we typically deploy configuration at least daily and sometimes several times a day, and we are working toward continuous configuration deployment.

Microservices in Strato may be configured using components that capture common patterns of infrastructure at Twitter, such as accessing data in Twitter's storage layer, Manhattan [15], or calling external Thrift services. Components may be parameterized with StratoQL expressions (for example, to validate a data structure before writing it to storage); or entire microservices may be defined with StratoQL (for example, to call several other microservices and manipulate the results). StratoQL is also used as a syntax to express configuration (it is run at configuration loading time).

Operations on microservices are described by types (see Figure 3.1). These types are used to generate Thrift, REST, and GraphQL [7] interfaces for accessing microservices, and an internal web interface for browsing them. Because Strato must interoperate with external systems that use type definitions written in Thrift IDL, we generate Strato types from Thrift IDL as part of configuration deployment. The collection of available operations and their types is organized into a *catalog*, analogous to the system catalog in a SQL database.

## 3 StratoQL

```
userByName.fetch("jack") match {
  case { userId = uId } =>
    for {
      { tweetId = tId, authorId = aId } <-
          timelineByUserId.fetch(uId)
      { text = text } = tweetById.fetch(tId)
      { userName = userName } = userById.fetch(aId)
    } yield (userName, text)
}
```

**Figure 1.** StratoQL

See Figure 1 for a representative StratoQL expression, which fetches a user record by username, fetches the user's Twitter timeline, then for each entry in the timeline fetches the tweet text and the author's username.

In the for-expression over timeline entries, the body expression is run for all elements concurrently, so the individual calls to the `tweetById` and `userById` microservices are run concurrently, and are batched into a single RPC call to each underlying service (see Section 4). The syntax `{ userId = uId }` is a record pattern (as found in OCaml and other languages), matching the `userId` field of a record value and binding its value to `uId`; the record may have additional unmentioned fields.

StratoQL has a Scala-like syntax (and a similar approach to typechecking, see Section 3.3), but differs from Scala in several ways:

In contrast to Scala's nominal type system, StratoQL's type system is purely structural. Structural typing is a good fit for our intended domain: because StratoQL code deals mostly with external data (from storage or other services), types must be meaningful across different services and different versions of the same service. For a similar reason, Thrift RPC interfaces are effectively structurally-typed: there is no requirement that a client and server agree exactly on their interface (when they are deployed separately it is common that they do not agree) so long as the different interfaces are *wire-compatible*—that is, all serialized values of one interface are valid serialized values of another. Thrift's serialization format is designed so that wire-compatibility is similar to structural subtyping—for example it permits a wider record to be passed where a narrower one is expected.

StratoQL provides native access to Thrift data: Thrift types can be represented directly as Strato types (in particular `structs` become records and `unions`/`enums` become variants), and StratoQL provides syntax for constructing and pattern-matching such types. Structural typing avoids the

need to import or refer to external Thrift type definitions in application code; developers can write down expressions of the desired type directly. Together these features make it very convenient to work with Thrift data in StratoQL. In contrast, working with Thrift data from Scala can be somewhat awkward. Scrooge [17], Twitter's tool for generating Scala bindings from Thrift IDL, generates Scala case classes and sealed traits from Thrift `structs` and `unions`. Scala doesn't support pattern matching by field name in case classes, so decomposing large, complicated Thrift structures is more difficult. And requiring imports and namespacing makes application code noisier.

Another departure from Scala is that concurrency in StratoQL is implicit; for example in an expression $(e_1, e_2)$, $e_1$ and $e_2$ are evaluated concurrently. This makes it convenient to compose calls to several services in a direct style, in contrast to concurrency libraries with monadic or applicative interfaces such as Twitter's Future [6].

Finally, StratoQL is much simpler than Scala: it does not support any of the object-oriented or modular aspects of Scala or any of Scala's fancier type-system features. For our target domain we don't expect users to write very large or complicated programs; and we have little need to interoperate with JVM code, so it is not necessary to support the full range of JVM types.

### 3.1 Types

$$
\begin{array}{rcl}
\tau & ::= & x \\
 & | & \mathsf{Unit} \mid \mathsf{Byte} \mid \mathsf{Boolean} \mid \mathsf{Int} \mid \mathsf{Double} \mid \mathsf{String} \mid \ldots \\
 & | & (\tau_1, \ldots, \tau_n) \\
 & | & \{\, l_1 \colon \tau_1, \ldots, l_n \colon \tau_n \,\} \\
 & | & \langle\, C_1(\tau_1) \mid \ldots \mid C_n(\tau_n) \,\rangle \\
 & | & \mathsf{Option}[\tau] \mid \mathsf{Seq}[\tau] \mid \mathsf{Set}[\tau] \mid \mathsf{Map}[\tau_k, \tau_v] \\
 & | & \mu x.\tau \\
 & | & X \\
 & | & \mathsf{Any} \\
 & | & \mathsf{Nothing} \\
 & | & \forall X_1, \ldots, X_m.(\tau_1, \ldots, \tau_n) \rightarrow \tau
\end{array}
$$

**Figure 2.** Types

Strato types (see Figure 2) that represent concrete, serializeable data are similar to Thrift types: they include primitive types, tuples, records (similar to Thrift `structs`), variants (subsuming Thrift `unions` and `enums`), a fixed set of container types (Option, Seq, Set, and Map) and recursive types.

In addition Strato supports type parameters, top and bottom elements, and type-parameterized functions; these types are used in typechecking but do not represent concrete, serializeable data.

Types are enriched with annotations that control special treatment in particular settings; e.g. the Thrift field tag for binary serialization, or a default value to use when a field is missing at deserialization time.

### 3.2 Syntax

$$
\begin{array}{rcl}
e & ::= & x \\
 & | & () \mid \mathsf{true} \mid 7 \mid \texttt{"foo"} \mid \ldots \\
 & | & (e_1, \ldots, e_n) \\
 & | & \{\, l_1 = e_1, \ldots, l_n = e_n \,\} \\
 & | & C \mid C(e) \\
 & | & \mathsf{Seq}(e_1, \ldots, e_n) \mid \mathsf{Set}(e_1, \ldots, e_n) \\
 & | & \mathsf{Map}(e_{k1} \texttt{->} e_{v1}, \ldots, e_{kn} \texttt{->} e_{vn}) \\
 & | & \{\, (x_1 \colon \tau_1, \ldots, x_n \colon \tau_n) \texttt{=>} e \,\} \\
 & | & \{\, (x_1, \ldots, x_n) \texttt{=>} e \,\} \\
 & | & e_1 + e_2 \mid e_1 \texttt{==} e_2 \mid \ldots \\
 & | & e.l \\
 & | & e.m(e_1, \ldots, e_n) \\
 & | & e.m[\tau_1, \ldots, \tau_m](e_1, \ldots, e_n) \\
 & | & \mathsf{val}\ p = e_p; e \\
 & | & e\ \mathsf{match}\ \{\, \mathsf{case}\ p_1 \texttt{=>} e_1 \ \ldots\ \mathsf{case}\ p_n \texttt{=>} e_n \,\} \\
 & | & \mathsf{for}\ \{\, x_1 \texttt{<-} e_1 \ \ldots \,\}\ \mathsf{yield}\ e \\
\\
p & ::= & x \mid _ \\
 & | & () \mid \mathsf{true} \mid 7 \mid \texttt{"foo"} \mid \ldots \\
 & | & (p_1, \ldots, p_n) \\
 & | & \{\, l_1 = p_1, \ldots, l_n = p_n \,\} \\
 & | & C \mid C(p)
\end{array}
$$

**Figure 3.** Expressions

See figure 3 for StratoQL syntax, comprising primitive values, tuples, records, variant arms, collections, functions (with and without argument type annotations), arithmetic and logical operators, field projections and method applications (with and without type parameter instantiations), binding, pattern matching, and for-expressions.

### 3.3 Typechecking

We follow Scala in implementing Pierce and Turner's local type inference [13], which involves three main algorithms:

- *synthesizing* a type from an expression when we don't know what type to expect;
- *checking* an expression against an expected type; and
- *subtyping* when synthesis meets checking.

as well as an algorithm to determine type parameters in applications of parameterized functions by solving local constraints.

To implement structural subtyping with recursive types we follow Amadio and Cardelli [2]: we maintain a context of subtyping assumptions (Amadio and Cardelli call it a "trail") involving recursive type bindings, in order to avoid looping when traversing recursive types (TRAIL in Figure 6).

$$\textsc{var}_e\ \frac{\Gamma_e(x) = \tau}{\Gamma_e \vdash x \overrightarrow{\in} \tau \Rightarrow x}$$

$$\textsc{val}\ \frac{\Gamma_e \vdash e_p \overrightarrow{\in} \tau_p \Rightarrow e_p' \qquad \cdot \vdash p \overleftarrow{\in} \tau_p \Rightarrow \Gamma_p \qquad \Gamma_e + \Gamma_p \vdash e \overrightarrow{\in} \tau \Rightarrow e'}{\Gamma_e \vdash (\mathsf{val}\ p = e_p; e) \overrightarrow{\in} \tau \Rightarrow (\mathsf{val}\ p = e_p'; e')}$$

$$\textsc{variant}\ \frac{\Gamma_e \vdash e \overrightarrow{\in} \tau \Rightarrow e'}{\Gamma_e \vdash C(e) \overrightarrow{\in} \langle\, C(\tau)\,\rangle \Rightarrow C(e')}$$

$$\textsc{record}\ \frac{\Gamma_e \vdash e_1 \overrightarrow{\in} \tau_1 \Rightarrow e_1' \qquad \ldots \qquad \Gamma_e \vdash e_n \overrightarrow{\in} \tau_n \Rightarrow e_n'}{\Gamma_e \vdash \{\, l_1 = e_1,\ \ldots, l_n = e_n \,\} \overrightarrow{\in} \{\, l_1 : \tau_1,\ \ldots,\ l_n : \tau_n \,\} \Rightarrow \{\, l_1 = e_1',\ \ldots, l_n = e_n' \,\}}$$

$$\textsc{apply}\ \frac{\Gamma_e(f) = \forall X_1, \ldots, X_m.(\tau_1, \ldots, \tau_n) \to \tau \qquad \cdot; \Gamma_e; \cdot \vdash e_1 \overleftarrow{\in} \tau_1 \Rightarrow e_1'; \theta_1 \qquad \ldots \qquad \theta_{n-1}; \Gamma_e; \cdot \vdash e_n \overleftarrow{\in} \tau_n \Rightarrow e_n'; \theta_n \qquad \sigma = \sigma_{\theta\tau}}{\Gamma_e \vdash f(e_1, \ldots, e_n) \overrightarrow{\in} \sigma\tau \Rightarrow \sigma(f[X_1, \ldots, X_m](e_1', \ldots, e_n'))}$$

**Figure 4.** Synthesis

We follow Odersky [12] in extending type argument synthesis to handle unannotated method arguments (e.g. `seq.map { x => x + 1 }`), by checking function arguments under a set of type parameter constraints (APPLY in Figure 4) rather than synthesizing function arguments and solving constraints at the point of application.

Our algorithm differs from these forebears in two ways:

- we compute an asynchrony effect for expressions (see Section 3.5), and support polymorphic asynchrony and asynchrony inference in function application; and
- we extend the checking algorithm to encompass type conversion when an expression is not a subtype of the expected type, but may be usefully converted to the expected type (see Section 3.4).

### 3.3.1 Synthesis

The synthesis algorithm

$$\Gamma_e \vdash e \overrightarrow{\in} \tau \Rightarrow e'$$

relates

- an expression $e$, which is elaborated to $e'$ with missing types filled in;
- a type $\tau$ synthesized from $e$; and
- a context $\Gamma_e$ binding expression variables in $e$.

with rules (see Figure 4)

$\textsc{var}_e$
: synthesize a type for a variable by looking it up in the environment.

VAL
: synthesize a type for a val binding by synthesizing a type for the bound expression $e_p$ then synthesizing a type for the body $e$ in an environment enriched with the pattern bindings. An auxiliary algorithm $\Gamma \vdash p \overleftarrow{\in} \tau \Rightarrow \Gamma'$ (not shown) is used to check the pattern against the synthesized type and extract the binding types. The MATCH rule is similar (not shown).

VARIANT
: synthesize a type for a variant expression by synthesizing a type for the associated expression.

RECORD
: synthesize a type for a record expression by synthesizing a type for each field.

APPLY
: synthesize a type for a polymorphic function application by checking argument expressions to generate type parameter constraints, then instantiating constraints $\theta$ to get a type parameter substitution $\sigma$ (following Section 3.5 of Pierce and Turner [13]).

and also structural rules (TUPLE, SEQ, etc.) and rules for primitives and operators (not shown). Types of microservice operations are looked up in the catalog.

### 3.3.2 Checking

The checking algorithm

$$\theta; \Gamma_e; \Gamma_\tau \vdash e \overleftarrow{\in} \tau \Rightarrow e'; \theta'$$

relates

- an expression $e$, which is elaborated to $e'$ with missing types filled in;
- a type $\tau$ against which $e$ is checked;
- contexts $\Gamma_e$ and $\Gamma_\tau$ binding expression variables in $e$ and type variables in $\tau$ respectively; and
- type parameter constraints $\theta$, which may be updated to $\theta'$.

with rules (see Figure 5)

REC
: check an expression against a recursive type by adding the type variable to the type environment then checking against the body.

$$\textsc{rec}\quad \frac{\theta; \Gamma_e; \Gamma_\tau + (x : \mu x.\tau) \vdash e \overleftarrow{\in} \tau \Rightarrow e'; \theta'}{\theta; \Gamma_e; \Gamma_\tau \vdash e \overleftarrow{\in} \mu x.\tau \Rightarrow e'; \theta'}$$

$$\textsc{var}_\tau\quad \frac{\Gamma_\tau(x) = \tau \qquad \theta; \Gamma_e; \Gamma_\tau \vdash e \overleftarrow{\in} \tau \Rightarrow e'; \theta'}{\theta; \Gamma_e; \Gamma_\tau \vdash e \overleftarrow{\in} x \Rightarrow e'; \theta'}$$

$$\textsc{variant}\quad \frac{\theta; \Gamma_e; \Gamma_\tau \vdash e \overleftarrow{\in} \tau_k \Rightarrow e'; \theta'}{\theta; \Gamma_e; \Gamma_\tau \vdash C_k(e) \overleftarrow{\in} \langle\, C_k(\tau_k) \mid \ \ldots \,\rangle \Rightarrow C_k(e'); \theta'}$$

$$\textsc{record}\quad \frac{\theta; \Gamma_e; \Gamma_\tau \vdash e_1 \overleftarrow{\in} \tau_1 \Rightarrow e_1'; \theta_1 \qquad \ldots \qquad \theta_{n-1}; \Gamma_e; \Gamma_\tau \vdash e_n \overleftarrow{\in} \tau_n \Rightarrow e_n'; \theta_n}{\theta; \Gamma_e; \Gamma_\tau \vdash \{\, l_1 = e_1,\ \ldots, l_n = e_n,\ \ldots \,\} \overleftarrow{\in} \{\, l_1 : \tau_1,\ \ldots,\ l_n : \tau_n \,\} \Rightarrow \{\, l_1 = e_1',\ \ldots, l_n = e_n' \,\}; \theta_n}$$

$$\textsc{function}\quad \frac{\theta; \Gamma_e + (x_1 : \tau_1, \ldots, x_n : \tau_n); \Gamma_\tau \vdash e \overleftarrow{\in} \tau \Rightarrow e'; \theta'}{\theta; \Gamma_e; \Gamma_\tau \vdash \{\, (x_1,\ \ldots,\ x_n) \Rightarrow e \,\} \overleftarrow{\in} (\tau_1,\ \ldots,\ \tau_n) \ \rightarrow \tau \Rightarrow \{\, (x_1 : \tau_1,\ \ldots,\ x_n : \tau_n) \Rightarrow e' \,\}; \theta'}$$

$$\textsc{val}\quad \frac{\Gamma_e \vdash e_p \overrightarrow{\in} \tau_p \Rightarrow e_p' \qquad \cdot \vdash p \overleftarrow{\in} \tau_p \Rightarrow \Gamma_p \qquad \theta; \Gamma_e + \Gamma_p; \Gamma_\tau \vdash e \overleftarrow{\in} \tau \Rightarrow e'; \theta'}{\theta; \Gamma_e; \Gamma_\tau \vdash (\mathsf{val}\ p = e_p; e) \overleftarrow{\in} \tau \Rightarrow (\mathsf{val}\ p = e_p'; e'); \theta'}$$

$$\textsc{synth}\quad \frac{\Gamma_e \vdash e \overrightarrow{\in} \tau_s \Rightarrow e' \qquad \theta; \cdot; \Gamma_\tau; \Sigma \vdash \tau_s <: \tau \Rightarrow \theta'}{\theta; \Gamma_e; \Gamma_\tau \vdash e \overleftarrow{\in} \tau \Rightarrow e'; \theta'}$$

**Figure 5.** Checking

$\textsc{var}_\tau$
: check an expression against a type variable by checking against the variable's binding in the type environment.

VARIANT
: check a variant expression against a variant type by finding the corresponding label in the type and checking the associated expression against the associated type; extra arms in the type are ignored.

RECORD
: check a record expression against a record type by checking each field in the expression against the corresponding field in the type; extra fields in the expression are ignored.

FUNCTION
: check a function expression without argument type annotations against a function type by checking a type for the function body in an environment enriched with the argument type bindings; elaborate the expression with argument type annotations.

VAL
: check a val binding by synthesizing a type for the bound expression $e_p$ then checking the body $e$ in an environment enriched with the pattern bindings. The MATCH rule is similar (not shown).

SYNTH
: (if no other rule applies) check an expression against a type by synthesizing a type for the expression then checking that the synthesized type is a subtype of the expected type. (In particular this rule applies when checking against a type parameter, so the synthesized type constrains the parameter.)

and also structural rules (TUPLE, SEQ, etc.) and rules for primitives (not shown).

#### 3.3.3 Subtyping

The subtyping algorithm

$$\theta; \Gamma_a; \Gamma_b; \Sigma \vdash \tau_a <: \tau_b \Rightarrow \theta'$$

relates

- types $\tau_a$ and $\tau_b$;
- contexts $\Gamma_a$ and $\Gamma_b$ binding type variables in $\tau_a$ and $\tau_b$ respectively;
- a context $\Sigma$ of subtyping assumptions $\tau_a <: \tau_b$; and
- type parameter constraints $\theta$, which may be updated to $\theta'$.

$$\textsc{trail}\quad \frac{(\tau_a <: \tau_b) \in \theta}{\theta; \Gamma_a; \Gamma_b; \Sigma \vdash \tau_a <: \tau_b \Rightarrow \theta}$$

$$\textsc{param}_{\mathrm{A}}\quad \frac{\theta \vdash X <: \tau_b \Rightarrow \theta'}{\theta; \Gamma_a; \Gamma_b; \Sigma \vdash X <: \tau_b \Rightarrow \theta'}$$

$$\textsc{rec}_{\mathrm{A}}\quad \frac{\theta; \Gamma_a + (x : \mu x.\tau_a); \Gamma_b; \Sigma + (\mu x.\tau_a <: \tau_b) \vdash \tau_a <: \tau_b \Rightarrow \theta'}{\theta; \Gamma_a; \Gamma_b; \Sigma \vdash \mu x.\tau_a <: \tau_b \Rightarrow \theta'}$$

$$\textsc{var}_{\mathrm{A}}\quad \frac{\Gamma_a(x) = \tau_a \qquad \theta; \Gamma_a; \Gamma_b; \Sigma \vdash \tau_a <: \tau_b \Rightarrow \theta'}{\theta; \Gamma_a; \Gamma_b; \Sigma \vdash x <: \tau_b \Rightarrow \theta'}$$

$$\textsc{variant}\quad \frac{\theta; \Gamma_a; \Gamma_b; \Sigma \vdash \tau_{a_1} <: \tau_{b_1} \Rightarrow \theta_1 \qquad \ldots \qquad \theta_{n-1}; \Gamma_a; \Gamma_b; \Sigma \vdash \tau_{a_n} <: \tau_{b_n} \Rightarrow \theta_n}{\theta; \Gamma_a; \Gamma_b; \Sigma \vdash \langle\, C_1(\tau_{a_1}) \mid \ldots \mid C_n(\tau_{a_n}) \,\rangle <: \langle\, C_1(\tau_{b_1}) \mid \ldots \mid C_n(\tau_{b_n}) \mid \ldots \,\rangle \Rightarrow \theta_n}$$

$$\textsc{record}\quad \frac{\theta; \Gamma_a; \Gamma_b; \Sigma \vdash \tau_{a_1} <: \tau_{b_1} \Rightarrow \theta_1 \qquad \ldots \qquad \theta_{n-1}; \Gamma_a; \Gamma_b; \Sigma \vdash \tau_{a_n} <: \tau_{b_n} \Rightarrow \theta_n}{\theta; \Gamma_a; \Gamma_b; \Sigma \vdash \{\, l_1 \colon \tau_{a_1}, \ \ldots, \ l_n \colon \tau_{a_n}, \ \ldots \,\} <: \{\, l_1 \colon \tau_{b_1}, \ \ldots, \ l_n \colon \tau_{b_n} \,\} \Rightarrow \theta_n}$$

**Figure 6.** Subtyping

with rules (see Figure 6)

TRAIL
: use subtyping assumptions in the trail to cut off recursion. TRAIL takes precendence over all other rules.

PARAM$_{\mathrm{A}}$
: update a type parameter constraint; there is a symmetric PARAM$_{\mathrm{B}}$ (not shown). It is an invariant that type parameters do not appear on both sides of <: at once.

REC$_{\mathrm{A}}$
: handle a recursive type by adding its bound variable to the type environment, and adding the current pair of types to the trail; there is a symmetric REC$_{\mathrm{B}}$ (not shown).

VAR$_{\mathrm{A}}$
: handle a type variable by consulting the environment; there is a symmetric VAR$_{\mathrm{B}}$ (not shown).

VARIANT
: handle variants by checking subtyping for the types associated with each label; also permit a narrower variant where a wider one is expected.

RECORD
: handle records by checking subtyping for the types associated with each label; also permit a wider record where a narrower one is expected.

and also structural rules (TUPLE, SEQ, etc.) and rules for primitives (not shown).

Type parameter constraint sets $\theta$ consist of upper and lower bounds $\tau_a <: X <: \tau_b$ for a set of type parameters. Updating the upper bound of a parameter $\theta \vdash X <: \tau_{b'} \Rightarrow \theta'$ computes the least upper bound of the previous upper bound $\tau_b$ and the new upper bound $\tau_{b'}$ (and similarly for lower bounds); update fails if the resulting constraint is unsatisfiable.

### 3.4 Conversion

A convenient feature of Thrift IDL is that it supports default values for `struct` fields. Default values provide for wire-compatibility: a `struct` extended with a field that has a default value is wire-compatible with the original `struct` in both directions—a receiver expecting the original `struct` can ignore the extra field, while a receiver expecting the extended `struct` can fill in the default value. It's common for Thrift structs to have many fields with default values, reflecting their evolution over time. We support default field values in Strato by annotating field types with a default value[1].

Record field defaults are also useful in application code; they are supported in Scala with argument defaults on case class constructors. In StratoQL records are not constructed with named constructors; however we can achieve much the same effect through *conversion*: when checking a record against a type, if the record lacks a field that is present in the type, we convert the record to the expected type, filling in the default.

By the same mechanism we convert narrower to wider numeric types. We also do conversions that make it easier to update the types of configuration objects (since code and configuration are deployed separately there is a need for backward compatibility). For example a variant expression Foo may be converted to type $\langle\, \mathsf{Foo}(\{\mathsf{bar}\colon \mathsf{Boolean} = \mathsf{true}\}) \,\rangle$ by filling in the default value of the record.

[1]Thrift IDL also supports optional fields; Strato conflates them to fields of Option type with default None.

$$\textsc{variant}\quad \frac{\theta;\Gamma_a;\Gamma_b;\Sigma \vdash \{\} \underset{\rightsquigarrow}{<:} e \Rightarrow \theta'}{\theta;\Gamma_a;\Gamma_b;\Sigma \vdash C \underset{\rightsquigarrow}{<:} C(e) \Rightarrow \theta'} \qquad \frac{\textsc{Int} \rightsquigarrow \textsc{Long}}{\theta;\Gamma_a;\Gamma_b;\Sigma \vdash \mathsf{Int} \underset{\rightsquigarrow}{<:} \mathsf{Long} \Rightarrow \theta'} \qquad \textsc{synth}_{\textsc{convert}}\quad \frac{\Gamma_e \vdash e \overset{\rightarrow}{\in} \tau_s \Rightarrow e' \qquad \theta;\cdot;\Gamma_\tau;\Sigma \vdash \tau_s \underset{\rightsquigarrow}{<:} \tau \Rightarrow \theta'}{\theta;\Gamma_e;\Gamma_\tau \vdash e \overset{\leftarrow}{\in} \tau \Rightarrow \mathsf{convert}_{\tau_s,\tau}(e');\theta'}$$

**Figure 7.** Conversion

$$\textsc{tuple}_{\leftarrow}\quad \frac{\theta;\Gamma_e;\Gamma_\tau \vdash e_1 \underset{\alpha}{\overset{\leftarrow}{\in}} \tau_1 \Rightarrow e_1';\theta' \qquad \ldots \qquad \theta;\Gamma_e;\Gamma_\tau \vdash e_n \underset{\alpha}{\overset{\leftarrow}{\in}} \tau_n \Rightarrow e_n';\theta'}{\theta;\Gamma_e;\Gamma_\tau \vdash (e_1,\ \ldots,\ e_n) \underset{\alpha}{\overset{\leftarrow}{\in}} (\tau_1,\ \ldots,\ \tau_n) \Rightarrow (e_1',\ \ldots,\ e_n');\theta'} \qquad \textsc{tuple}_{\rightarrow}\quad \frac{\Gamma_e \vdash e_1 \underset{\alpha_1}{\overset{\rightarrow}{\in}} \tau_1 \Rightarrow e_1' \qquad \ldots \qquad \Gamma_e \vdash e_n \underset{\alpha_n}{\overset{\rightarrow}{\in}} \tau_n \Rightarrow e_n'}{\Gamma_e \vdash (e_1,\ \ldots,\ e_n) \overset{\rightarrow}{\in} \underset{\bigvee_i \alpha_i}{(\tau_1,\ \ldots,\ \tau_n)} \Rightarrow (e_1',\ \ldots,\ e_n')}$$

$$\textsc{function}\quad \frac{\theta;\Gamma_e + (x_1:\tau_1,\ldots,x_n:\tau_n);\Gamma_\tau \vdash e \underset{\alpha_f}{\overset{\leftarrow}{\in}} \tau \Rightarrow e';\theta'}{\theta;\Gamma_e;\Gamma_\tau \vdash \{(x_1,\ \ldots,\ x_n) \texttt{=>} e\} \underset{\alpha}{\overset{\leftarrow}{\in}} (\tau_1,\ \ldots,\ \tau_n) \underset{\alpha_f}{\rightarrow} \tau \Rightarrow \{(x_1{:}\ \tau_1,\ \ldots,\ x_n{:}\ \tau_n) \texttt{=>} e'\};\theta'}$$

$$\textsc{apply}\quad \frac{\begin{array}{c}\Gamma_e(f) = \forall A.\forall X_1,\ldots,X_m.(\tau_1,\ldots,\tau_n) \underset{A}{\rightarrow} \tau \\ \cdot;\Gamma_e;\cdot \vdash e_1 \underset{A}{\overset{\leftarrow}{\in}} \tau_1 \Rightarrow e_1';\theta_1 \qquad \ldots \qquad \theta_{n-1};\Gamma_e;\cdot \vdash e_n \underset{A}{\overset{\leftarrow}{\in}} \tau_n \Rightarrow e_n';\theta_n \qquad \sigma = \sigma_{\theta\tau}\end{array}}{\Gamma_e \vdash f(e_1,\ldots,e_n) \underset{\sigma A}{\overset{\rightarrow}{\in}} \sigma\tau \Rightarrow \sigma(f[X_1,\ldots,X_m](e_1',\ldots,e_n'))}$$

$$\textsc{synth}\quad \frac{\Gamma_e \vdash e \underset{\alpha_s}{\overset{\rightarrow}{\in}} \tau_s \Rightarrow e' \qquad \theta;\cdot;\Gamma_\tau;\Sigma \vdash \tau_s <: \tau \Rightarrow \theta' \qquad \theta' \vdash \alpha_s <: \alpha \Rightarrow \theta''}{\theta;\Gamma_e;\Gamma_\tau \vdash e \underset{\alpha}{\overset{\leftarrow}{\in}} \tau \Rightarrow e';\theta''}$$

**Figure 8.** Asynchrony

The subtyping and checking algorithms extend naturally to support conversion (see Figure 7). We extend the expression syntax with an element $\mathsf{convert}_{\tau,\tau'}$ indicating conversion, and elaborate expressions using this element when conversion is needed during typechecking (in the $\textsc{synth}_{\textsc{convert}}$ rule). The convertibility algorithm

$$\theta;\Gamma_a;\Gamma_b;\Sigma \vdash \tau_a \underset{\rightsquigarrow}{<:} \tau_b \Rightarrow \theta'$$

extends the subtyping algorithm with rules for various conversions; $\Sigma$ becomes a context of convertibility rather than subtyping assumptions.

### 3.5 Asynchrony

Because concurrency in StratoQL is implicit, whether a StratoQL expression is asynchronous is not revealed by its type, in contrast to concurrency libraries with monadic or applicative interfaces [6] [10].

However it is often useful to know whether an expression is asynchronous. For example, Strato configuration files must be synchronous; in match expressions pattern guards must be synchronous (because they compile to `choose` calls, see Section 4.2); and when compiling an expression we could save some overhead if we know it's synchronous (we do not currently implement this optimization).

To determine asynchrony of StratoQL expressions, we extend the synthesis and checking algorithms to compute an asynchrony *effect* [11] $\alpha$ along with the type. Function types carry an asynchrony flag to indicate that an application of the function has the given asynchrony.

There is a natural notion of sub-asynchrony: a synchronous expression may appear where an asynchronous expression is expected (but not the other way around); similarly a function of synchronous type may appear where a function of asynchronous type is expected.

For applications of higher-order functions, we'd like to consider the asynchrony of the argument functions, so e.g. map can be used in a synchronous context if its argument function has synchronous type. Therefore we implement asynchrony polymorphism (for a single parameter $A$) and

inference as for types; we track constraints in $\theta$ and extend sub-asynchrony to constrain $A$.

The rule modifications are mostly straightforward (see Figure 8):

TUPLE$_{\leftarrow}$
: check a tuple against $\alpha$ by checking each component against $\alpha$.

TUPLE$_{\rightarrow}$
: synthesize $\alpha$ for a tuple by synthesizing $\alpha_i$ for each component, returning true if any $\alpha_i$ is true.

FUNCTION
: check the function body against $\alpha_f$ (asynchrony of the function type); a function literal is synchronous so $\alpha$ is ignored.

APPLY
: synthesize $\alpha$ for an asynchrony-polymorphic function application by checking argument expressions to generate asynchrony parameter constraints, and instantiate them as for types. Types $\tau_i$ may contain the asynchrony parameter, which is constrained by checking a function against $\tau_i$.

SYNTH
: check $\alpha$ for an expression by synthesizing $\alpha_s$ for the expression then checking that $\alpha_s$ is a sub-asynchrony of $\alpha$; the sub-asynchrony check $\theta' \vdash \alpha_s <: \alpha \Rightarrow \theta''$ may update constraints on the asynchrony parameter as with type parameters.

# 4 Stitch

```
// Haxl
friendsOf :: UserId -> Fetch [UserId]

length <$> intersect' (friendsOf x) (friendsOf y)
  where intersect' = liftA2 intersect

// Stitch
def friendsOf(id: UserId): Stitch[Seq[UserId]]

join(friendsOf(x), friendsOf(y))
  .map { case (a, b) =>
    a.intersect(b).length
  }
```

**Figure 9.** Stitch

In order to support StratoQL's implicit concurrency, we compile StratoQL expressions into calls to a Scala concurrency library called Stitch. Stitch was inspired by Facebook's Haxl library [10], and supports a similar monadic / applicative interface. For example, Figure 9 shows a translation of an example from Marlow [10]; here `join(a, b)` is roughly `(,) <$> a <*> b` in Haskell terms.

Like Haxl, Stitch provides atomic computations that represent calls to underlying services, and a set of combinators to compose larger computations. We represent a computation as a syntax tree (e.g. a `join` becomes a `Join` node); to evaluate a computation we

- traverse the syntax tree to find atomic calls;
- deduplicate and batch the atomic calls;
- invoke underlying services with batches of calls;
- wait for some underlying service RPC to return;
- substitute the results of calls into the tree;
- simplify the tree; and
- repeat until the tree represents a constant value.

Stitch predates Strato by a few years; it's used in tens of existing services at Twitter.

## 4.1 Stitch Arrows

```
sealed trait Arrow[-T, +U]

def identity[T]: Arrow[T, T]
def value[T](v: T): Arrow[Any, T]
def call[T, U](s: Service[T, U]): Arrow[T, U]
def map[T, U](f: T => U): Arrow[T, U]

def join[T, U, W](a1: Arrow[T, U], a2: Arrow[T, W]):
  Arrow[T, (U, W)]

def andThen[T, U, W](a1: Arrow[T, U], a2: Arrow[U, W]):
  Arrow[T, W]

def choose[T, U, V](
  choices: (PartialFunction[T, U], Arrow[U, V])*
): Arrow[T, V]

def sequence[T, U](a: Arrow[T, U]):
  Arrow[Seq[T], Seq[U]]

def traverse[T, U](a: Arrow[T, U], ts: Seq[T]):
  Stitch[Seq[U]]
```

**Figure 10.** Arrows

After some experience using Stitch in production systems, we discovered that its monadic / applicative interface has a performance cost:

A typical computation involves several `flatMap`s to capture data dependencies between service calls. To evaluate a `flatMap` we run a `Stitch[T]`-returning function, which constructs a syntax tree for the subcomputation. This interface permits subcomputations that depend dynamically on data fetched earlier in the computation. However, this is relatively rare in practice; most of the time a `flatMap` always builds a tree of the same shape, with data dependencies substituted into the tree in a uniform way. (For example the computation in Figure 1 is static in this way.) Rebuilding nearly-identical trees over and over is costly in memory allocation and CPU time.

To improve performance we turned to *arrows* [8]. An interface based on arrows gives us a way to build the tree representing a computation once, then evaluate it many times with different data. An Arrow[T, U] is similar to a T => Stitch[U], but the static structure of the computation it represents is visible to the Stitch implementation. So the implementation can avoid needless tree allocation, and perform other optimizations (e.g. folding constant arrows).

See Figure 10 for a sketch of the interface, which includes combinators

`identity`
: return input unchanged

`value`
: ignore input and return a constant value.

`call`
: send input to a service (with deduplication and batching), return its result.

`map`
: apply a function to input, return its result.

`join`
: send input to two arrows, return a tuple of their results.

`andThen`
: send input to `a1`, send its result to `a2`.

`choose`
: for each choice `(pf, a)` in `choices`, if `pf` is defined on input, send `pf(input)` to `a`, otherwise try the next choice

`sequence`
: send each element of input to `a` concurrently.

and `traverse`, which applies an arrow uniformly to a sequence, achieving our goal of reusing the static description of a computation.

While the arrows interface in Stitch provides better performance, it is much less convenient to use; in particular threading accumulated values through an arrow computations is tedious compared to using the lexical scope available to a monadic interface. (One service at Twitter was converted to use arrows for performance, then converted back to the monadic interface when it was decided that readability was worth the performance cost.) Fortunately compiling StratoQL to arrows provides both performance and convenience.

### 4.2 Compiling to Arrows

Following Hughes [8] (see Section 4.2), we compile a StratoQL expression to an arrow that takes a value environment as input and returns the value of the expression.

A tuple expression compiles to a `join` of the compilations of its elements, so the elements are evaluated concurrently; similarly a `for`-expression compiles to a `sequence`, so the body is evaluated concurrently for each element.

A pattern binding expression (match, val, or binding in a for-expression) compiles to a `choose` over the compilations of its cases. The partial function for each case checks that the pattern and guard are satisfied and updates the value environment with the extracted bindings. Because these are ordinary Scala partial functions, they must be synchronous, so we check that StratoQL expressions in pattern guards are synchronous (see Section 3.5).

### 4.3 Example

```
def lookup(id: String) = map { env => env(id) }
def pairWithArg(a: Arrow[T, U]) = join(identity, a)

val userByNameFetch = call(...)
val timelineByUserIdFetch = call(...)
val tweetByIdFetch = call(...)
val userByIdFetch = call(...)

val user = value("jack").andThen(userByNameFetch)
val userPat = [...] // { userId = uId }
val userArrow = {
  val timeline = lookup("uId")
    .andThen(timelineByUserIdFetch)
  val timelineEntryPat = [...]
    // { tweetId = tId, authorId = aId }
  val timelineEntryArrow = {
    val tweet = lookup("tId").andThen(tweetByIdFetch)
    val user = lookup("aId").andThen(userByIdFetch)
    pairWithArg(tweet).andThen(choose((
      tweetPat, // { text = text }
      pairWithArg(user).andThen(choose((
        userPat, // { userName = userName }
        join(lookup("userName"), lookup("text"))
      )))
    )))
  }
  pairWithArg(timeline)
    .map { case (env, seq) => seq.map { v => (env, v) }
    .andThen(sequence(
      choose((timelineEntryPat, timelineEntryArrow))
    ))
}
pairWithArg(user).andThen(
  choose((userPat, userArrow))
)
```

**Figure 11.** Compiled StratoQL

See Figure 11 for the compilation of Figure 1 into arrows. We use auxiliary functions `lookup` to look up bindings in the environment and `pairWithArg` to carry the environment along with the result of an arrow.

Atomic calls compile to uses of `call` arrows. As above, pattern bindings compile to `choose` calls. (Partial functions which extract the bindings for a pattern are not shown.)

The `sequence` function applies its argument arrow to each element of the input sequence concurrently, and concurrent atomic calls to the same underlying service are batched

together. So there are only 4 Thrift calls to underlying services: one each to `userByName`, `timelineByUserId`, `tweetById`, and `userById`.

(Note that the `tweetById`, and `userById` calls in the original query are independent and could be run concurrently, but our current compilation strategy for for-expressions doesn't make this optimization.)

## 5 Experience with Strato

The Strato system has been under development for 3 years, and taking production traffic for 2 years. Tens of teams use Strato, and there are hundreds of microservices running on the platform. Strato powers a new API under development for mobile clients based on GraphQL; eventually this will obviate the need for many existing HTTP services. And we are developing a system for processing event streams using hosted functions written in StratoQL.

It's clear that teams using Strato enjoy increased leverage: it saves them considerable effort, so enables them to deliver features more quickly, and experiment at lower risk; and because teams own and develop their own microservice configurations and can deploy them frequently, they retain a lot of autonomy.

A platform like Strato supporting flexible application logic seems to demand some kind of programming language in which users can express logic; however the choice to develop a new language is a complicated one.

On one hand, the requirement that teams use StratoQL costs them some autonomy: they must rely on the Strato team to add missing features, fix bugs, write documentation, provide support, provide tooling (such as IDE integration), etc., all of which would come "for free" with a stock language such as Scala. This has been a source of frustration for our users. StratoQL's support for implicit concurrency and native access to Thrift data are very convenient, but this is not the main leverage teams get from Strato; it is rather avoiding the fixed costs of running a service.

On the other hand, a bespoke language and implementation makes it much easier to provide a service platform, for several reasons:

- we don't need to invoke an unwieldy, expensive compiler designed for a general-purpose language in order to load configuration;
- we have more control over runtime loading of code since we're not tied to the JVM bytecode model;
- it's easier to support computation over types that are defined in configuration and can be updated at runtime;
- it's easier to limit the scope of what we must support and operate since we can leave out or limit expensive language features;
- the implementation is small and simple, and we can tailor it to our relatively narrow needs without the complexity burden of a general-purpose language implementation; and
- we can implement domain-specific optimizations (for example, avoiding deserializing data which is not actually accessed in the request path) which would be difficult in a general-purpose language implementation.

We're hopeful that as Strato and StratoQL mature, the Strato team will be less of a bottleneck for our users.

## 6 Related Work

There are several service platforms available through public cloud providers, such as AWS Lambda, Google Cloud Functions, and Azure Functions. They typically support mainstream languages such as Java or Javascript, and interface with other systems using arbitrary HTTP requests. They don't attempt to provide a catalog of available services or describe them with a uniform type system.

There are a large number of commercial "data virtualization" systems (e.g. Delphix, Denodo, and Gluent) that make multiple data sources available in a single catalog and provide a uniform way to query them using SQL. They are typically aimed at analytics use cases rather than high-scale application use cases, and don't typically support embedded application logic.

Haxl [10] shares many goals with Strato (particularly as it is used at Facebook [9]); however Haxl is a domain-specific language embedded in Haskell rather than an external DSL. Compared to Stitch, Haxl has only a monadic interface, and may be subject to the performance issues noted in Section 4.

Lightweight Modular Staging [14] is an approach to embedding domain-specific code generation in a way that reuses Scala syntax and typechecking. It does not address the issues noted above that make it challenging to embed a general purpose language implementation in a service platform.

FrTime [5] is a domain-specific language that supports implicit functional reactivity by compiling to an underlying library, much as StratoQL supports implicit concurrency by compiling to Stitch. It implements an overhead-reducing optimization similar to the one conjectured in Section 3.5.

## Acknowledgments

Strato has been the work of many people at Twitter: David Benjamin, Marius Eriksen, Erik Froese, Ryan Greenberg, Matthew Jeffryes, Alex Levenson, Runhang Li, Ran Magen, Benjamin Navetta, Jonathan Simms, and Michael Solomon.

We would like to thank Joy Su, who managed the Strato team since its formation. We would also like to thank the many Strato customers who have contributed feedback, bug reports, and code to Strato.

Finally we would like to thank the anonymous reviewers for their thorough and helpful comments.

## References


[1] Aditya Agarwal, Mark Slee, and Marc Kwiatkowski. 2007. *Thrift: Scalable Cross-Language Services Implementation.* Technical Report. Facebook. http://thrift.apache.org/static/files/thrift-20070401.pdf

[2] Roberto M. Amadio and Luca Cardelli. 1991. Subtyping Recursive Types. In *Proceedings of the 18th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '91).* ACM, New York, NY, USA, 104–118. https://doi.org/10.1145/99583.99600

[3] Apache Foundation. 2018. Aurora. https://github.com/apache/aurora/. (2018). [Online; accessed 21-May-2018].

[4] Apache Foundation. 2018. Mesos. https://github.com/apache/mesos. (2018). [Online; accessed 21-May-2018].

[5] Kimberley Burchett, Gregory H. Cooper, and Shriram Krishnamurthi. 2007. Lowering: A Static Optimization Technique for Transparent Functional Reactivity. In *Proceedings of the 2007 ACM SIGPLAN Symposium on Partial Evaluation and Semantics-based Program Manipulation (PEPM '07).* ACM, New York, NY, USA, 71–80. https://doi.org/10.1145/1244381.1244393

[6] Marius Eriksen. 2013. Your Server As a Function. In *Proceedings of the Seventh Workshop on Programming Languages and Operating Systems (PLOS '13).* ACM, New York, NY, USA, Article 5, 7 pages. https://doi.org/10.1145/2525528.2525538

[7] Facebook, Inc. 2016. GraphQL. http://facebook.github.io/graphql. (2016). [Online; accessed 21-May-2018].

[8] John Hughes. 2000. Generalising Monads to Arrows. *Sci. Comput. Program.* 37, 1-3 (May 2000), 67–111. https://doi.org/10.1016/S0167-6423(99)00023-4

[9] Simon Marlow. 2013. The Haxl Project at Facebook. https://skillsmatter.com/skillscasts/4429-simon-marlow. (2013). [Online; accessed 21-May-2018].

[10] Simon Marlow, Louis Brandy, Jonathan Coens, and Jon Purdy. 2014. There is No Fork: An Abstraction for Efficient, Concurrent, and Concise Data Access. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP '14).* ACM, New York, NY, USA, 325–337. https://doi.org/10.1145/2628136.2628144

[11] Flemming Nielson and Hanne Riis Nielson. 1999. Type and Effect Systems. In *Correct System Design, Recent Insight and Advances.* Springer-Verlag, Berlin, Heidelberg, 114–136. http://dl.acm.org/citation.cfm?id=646005.673740

[12] Martin Odersky, Christoph Zenger, and Matthias Zenger. 2001. Colored Local Type Inference. In *Proceedings of the 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '01).* ACM, New York, NY, USA, 41–53. https://doi.org/10.1145/360204.360207

[13] Benjamin C. Pierce and David N. Turner. 2000. Local Type Inference. *ACM Trans. Program. Lang. Syst.* 22, 1 (Jan. 2000), 1–44. https://doi.org/10.1145/345099.345100

[14] Tiark Rompf and Martin Odersky. 2010. Lightweight Modular Staging: A Pragmatic Approach to Runtime Code Generation and Compiled DSLs. *SIGPLAN Not.* 46, 2 (Oct. 2010), 127–136. https://doi.org/10.1145/1942788.1868314

[15] Peter Schuller. 2014. Manhattan, our real-time, multi-tenant distributed database for Twitter scale. https://blog.twitter.com/engineering/en%5Fus/a/2014/manhattan-our-real-time-multi-tenant-distributed-database-for-twitter-scale.html. (2014). [Online; accessed 21-May-2018].

[16] Twitter Engineering. 2011. Finagle: A Protocol-Agnostic RPC System. https://blog.twitter.com/engineering/en%5Fus/a/2011/finagle-a-protocol-agnostic-rpc-system.html. (2011). [Online; accessed 21-May-2018].

[17] Twitter, Inc. 2018. Scrooge. https://twitter.github.io/scrooge/. (2018). [Online; accessed 21-May-2018].